MASTER'S MODULE · AGENTIC AI · WINTER SEMESTER

Observability, Tracing & Debugging

Seeing what a nondeterministic agent actually did — and proving it afterwards.

Unit 07 · Part 2 of 2 · Session 11 · Week 11
Prof. Dr. Hauke Schramm · Department of Computer Science · Kiel University

Today's agenda

1. **The debugging problem** why nondeterminism defeats classical debugging
2. **What to record** traces, spans, payloads, tokens, cost, latency
3. **Tooling** LangSmith, Langfuse, OpenTelemetry GenAI conventions
4. **Reading a trace** a worked case, replay, regression tests
5. **Production** monitoring, privacy in traces, the cost of looking

Where we are

Unit 07, part 2 — we built the fence; now we need to see inside it

Last week (S10) — we bounded what the agent can do: sandboxes below, guardrails above

What the fence gives — a wrong action has limited cost — but the fence is opaque

The gap — a contained agent can still fail uselessly: loop, burn budget, mislead

This week — instruments to see, reconstruct, and debug every run after the fact

GUIDING QUESTION

When a run goes wrong, can you say afterwards exactly what happened — and prove it?

1

The debugging problem

Why nondeterminism breaks every classical instinct

Where the debugger stops helping

Debuggers remain useful for the runtime; the model's decisions need other instruments

Still debuggable — tool arguments, permissions, checkpoints, API contracts, state transitions

No reproduction — same input, different sample — the failure may simply not appear again

No stack trace — a bad decision is a successful function call; nothing throws

No breakpoints — the decision is a forward pass; you cannot step into weights

Delayed surfacing — the mistake at step 3 appears as nonsense at step 14

Consequence — traces and repeated evaluations add visibility into model-driven decisions

THE SHIFT

Deterministic debugging of the runtime plus statistical validation of behaviour: both, not either.

Debugging: classical vs agentic

The two complement each other: deterministic fixes for the runtime, statistical validation for behaviour

Classical debugging (runtime)

- rerun reproduces the crash
- stack trace points at the faulty line
- breakpoint, inspect state, step forward
- failure is an exception or wrong value
- a passing test confirms the fix within its coverage

Agent debugging (decisions)

- rerun samples a new trajectory
- failure is a plausible wrong choice
- runtime state is inspectable; the model's choice is not
- the run 'succeeds' and is still wrong
- a fix needs many sampled runs to verify

The uncomfortable truth

There is no stack trace for a bad decision.

A derailed run usually throws nothing: every call returns cleanly, the loop terminates, the report reads fluently. The defect is semantic — visible only if you recorded what the agent saw and chose at every step.

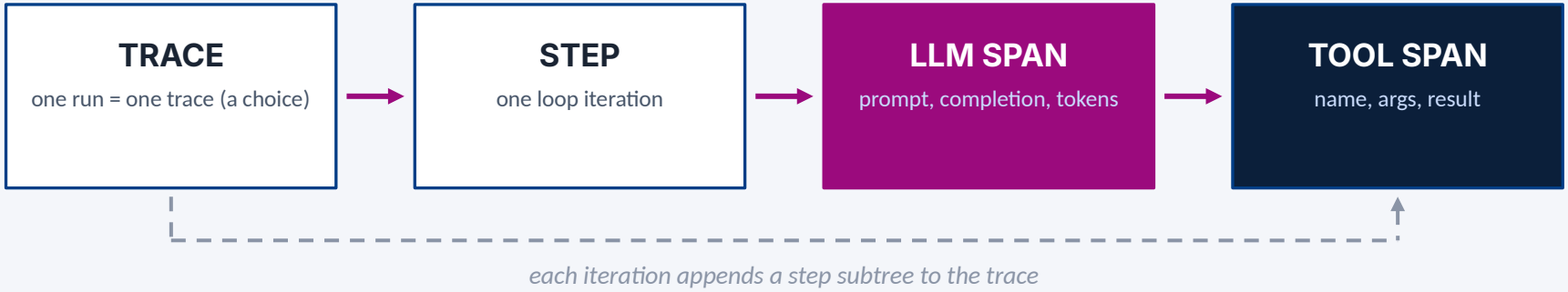
2

What to record

Traces, spans, and the three pillars adapted to agents

Anatomy of an agent trace

One run becomes a tree of timed, attributed spans



- A trace is a tree of spans sharing one trace ID — the model comes from Dapper (Sigelman et al., 2010).
- Parentage captures execution structure; span links and artifact IDs capture additional dependencies (fan-in, cross-agent).
- Attributes carry the payloads; timestamps carry the latency of every span.

What every span should carry

Sufficient evidence under an explicit capture policy: metadata by default, approved or redacted payloads where needed

Payloads (by policy) — prompt, completion, tool args and result — counts say that, payloads say what

Token accounting — map provider usage fields to disjoint billing categories; mark subset counters (cached, reasoning)

Cost — tokens times the tariff valid for that run — store the tariff, attribute to run, step, tool

Latency — monotonic clock per span; provider time vs tool time vs your code

Metadata — model ID, sampling parameters, prompt version, errors, retries

AUDIT ACTIONS

Audit observable actions, evidence, approvals and state changes. Reasoning summaries are supplementary evidence, not a faithful execution trace of the model (S05; Chen et al., 2025).

The three pillars, adapted to agents

Logs, traces, metrics — and which one leads here

Logs

Timestamped structured events. Cheap and greppable, but a flat stream loses the causal shape of a multi-step run.

Traces

The span tree per run: causality, payloads, the full trajectory. For agents, this is the primary artifact.

Metrics

Aggregates across runs: cost, latency, error rates. Feed dashboards and alerts; carry no per-run detail.

Tooling landscape (2026)

Compare by deployment, licence, operation and integration — the product name alone decides none of them

Option	Deployment / licence	Strengths	Watch out
LangSmith	managed SaaS or self-hosted (enterprise); closed core	deep LangChain tie-in; traces, evals, prompt hub	data location and operating duties follow deployment and contract
Langfuse	open source (MIT), self-hostable or cloud; part of ClickHouse since Jan 2026	framework-agnostic SDKs; cost tracking; evals	self-hosted: you operate and scale the backend
OTel GenAI	open standard, not a product	vendor-neutral gen_ai.* spans; any OTLP backend	all gen_ai.* still Development status; payloads off by default
Build your own	structured logs + database	full control; no new vendor or dependency	you rebuild search, diffing, and the UI

Decision in practice: a product for the UI and evals, the OTel standard for the instrumentation layer.

OpenTelemetry GenAI semantic conventions

The problem — every vendor invents its own trace schema — instrumentation becomes lock-in

OpenTelemetry — the CNCF standard for traces, metrics, logs; OTLP as the wire protocol

GenAI conventions — standard attribute names: gen_ai.request.model, gen_ai.usage.input_tokens

Scope — spans for inference calls, tool executions, and agent operations

Status 2026 — usable, but every gen_ai.* signal is still Development, not Stable; moved to its own repo in June 2026 — pin versions

WHY IT MATTERS

Instrument once against the standard; choose — or change — the backend later without touching agent code.

3

Reading a trace

A worked debugging case — and what replay buys you

Case study: the run that never ends

A web-research assistant (search, fetch, write tools) is given an obscure topic and hits its 40-step cap

Symptom — no output after 40 steps; the S10 budget cap fired and killed the run

Attempt 1: rerun — this time it finishes in 12 steps — nothing learned, bug still latent

Attempt 2: prints — megabytes of console text with no structure and no payload boundaries

Attempt 3: final context — early steps already summarised away by S08 compaction — evidence gone

The real problem — the evidence existed only at runtime and was never recorded

CONTAIN VS DIAGNOSE

The S10 guardrail limited the damage. It explained nothing. These are different jobs.

Instrumenting the tool boundary

One wrapper makes every tool call a recorded span (pseudo-API; Lab 11 builds the tracer itself in plain Python)

tracing.py

```
def traced(tool):
    def run(**args):
        span = tracer.span(tool.name,
            input=sanitize(args))
        t0 = time.perf_counter()
        try:
            out = tool(**args)
            span.set(output=sanitize(out))
            return out
        except Exception as e:
            span.set(error=repr(e))
            raise
        finally:
            dt = time.perf_counter() - t0
            safe_end(span, ms=1e3 * dt)
    return run
```

Capture policy — sanitize() applies it to args and result

Errors too — recorded, then re-raised unchanged

Export kept apart — safe_end logs its own failures; the tool status never changes

Limits — closing runs on Python exit paths; crash durability is separate

Reading the trace: diagnosis in three steps

The same stuck run, now with the flight recorder on

1 - Shape — the tree shows 38 near-identical step subtrees — a loop signature

2 - Zoom in — every search span returns the same value: a bare empty list

3 - Root cause — empty is a valid business result, not an error — but a bare [] carries no signal, and nothing stops the paraphrase-and-retry

Fix: feedback — return {"status": "no_results"} with guidance (S03 contract); a technical error stays a separate path

Fix: stop policy — the loop caps repeated no-result calls and allows an honest 'not found'; re-test — a model may ignore the message

TWO LAYERS, ONE FIX

Business result vs technical error is the tool's job to distinguish; bounding retries is the orchestration's. The trace made both visible in minutes.

Replay and regression tests from traces

Record once — a trace freezes one run into a replayable artifact; the next model call is still stochastic

Three modes — exact replay of recorded events; model re-decides against mocked tools; full live rerun

Matching — serve fixtures by tool name and arguments; report a miss — a new policy may call what was never recorded

Trajectory asserts — check properties: no repeated identical tool call; every claim cites a fetched source

Caveat — one passing replay proves little — sample several runs per case (S14)

A DATASET, WITH A BIAS

Production failures are realistic test cases, but failures alone skew the eval set. Keep random samples of apparently successful runs too.

4

Observability in production

Monitoring, privacy, and the cost of looking

Online monitoring: what should page you

Metrics over all runs — the aggregate view traces cannot give

Error rates

Tool failures, schema violations, guardrail denials per hour. A rising denial rate often means upstream content changed, not that users turned hostile.

Cost spikes

Cost per run and per user, alerted on deviation. A 10x spike is usually a loop or a context that quietly grew — catch it before the invoice does.

Latency drift

p50 and p99 per span kind. Slow tools stall whole loops; provider latency shifts under load and across model versions.

Behavioural drift

Steps per run, tool mix, refusal rate over time. The provider updates the model and your agent changes behaviour without any deploy on your side.

Privacy: traces are a liability too

Full-payload recording collides with data protection

What traces capture

- full user prompts — names, health, finances
- documents retrieved under licence terms
- secrets users paste into the chat
- internal records surfaced by tool calls
- all of it searchable, for every engineer

What governance requires

- capture policy: metadata default, payloads redacted
- redact at ingestion; one scrubbed email proves little
- retention and access rules, not debugging convenience
- synthetic data in development: dev traces leak too
- GDPR erasure reaches traces; hosted: DPA, region

The cost of observability itself

Storage — full payloads on every span — agent traces are megabytes, not log lines

Latency — synchronous export blocks the loop; batch and ship asynchronously

Sampling — head sampling drops rare failures; tail-based keeps errors and outliers — but not silent semantic ones

Attention — dashboards nobody reads and alerts everybody mutes are pure cost

Working rule — development: every event, synthetic data; production: keep failures, random-sample apparent successes

HONESTY

Observability is a feature you design, build, and pay for — budget it like one.

Lab 11: build the flight recorder yourself

A plain-Python tracer for the lab's offline search-read-write task — no backend, no API key (Parts A-G, 90-120 min)

Trace (B, D) — Span and Tracer classes: a tree of timed spans, one trace per run; JSONL persistence; render tree and token/cost account

Instrument (C) — traced(…) wrapper at the tool boundary; break it on the planted no-results topic

Diagnose (E) — a prepared faulty trace, no tool source: loop signature (R4 detector), tool-contract bug (R1)

Fix and prove (F) — explicit no-results contract; replay-based regression test asserting a trajectory property

Redact (F) — PII scrubber on the export path, verified with a seeded email address

DELIVERABLES

Report tasks R1-R4: three-sentence diagnosis, the cost of looking, replay and its limits, the loop detector. Ollama (qwen2.5:7b) only for one optional cell in Part G.

TAKEAWAYS

What you should remember

- **No stack trace —** agent failures are semantic; the debugger keeps the runtime, traces cover the decisions
- **Traces first —** a tree of spans per run: policy-governed payloads, tokens, cost, latency, structure
- **Audit actions —** actions, evidence, approvals, state changes; reasoning summaries only supplement
- **Read the shape —** loop signatures and repeated spans turn days of guessing into minutes
- **Replay —** three replay modes; recorded traces become regression tests and (biased) eval data
- **Production —** monitor errors, cost, drift; capture policy and redaction; budget the observability itself

NEXT SESSION Multi-agent systems and self-reflection.

Reading & references

- Sigelman, B. H. et al. (2010). Dapper, a Large-Scale Distributed Systems Tracing Infrastructure. Google Technical Report.
  The origin of the trace/span model that every modern tracing system inherits.
- Majors, C., Fong-Jones, L., Miranda, G. (2022). Observability Engineering. O'Reilly.
  High-cardinality, high-dimensionality events as the core of observability — beyond the three pillars.
- OpenTelemetry (2026). Semantic Conventions for Generative AI Systems. github.com/open-telemetry/semantic-conventions-genai.
  The gen_ai. attribute and span definitions; all still Development status as of 2026 — check the current stability status per signal.*
- Langfuse (2026). Langfuse Documentation. LangChain (2026). Self-hosted LangSmith overview. enterprise-hub.langchain.com/onboarding/self-hosted-overview.
  Two trace products as reference tools (Lab 11 builds its own tracer): Langfuse open source, part of ClickHouse since 2026; LangSmith managed or self-hosted.
- Chen, Y. et al. (2025). Reasoning Models Don't Always Say What They Think. arXiv:2505.05410.
  Why reasoning traces are supplementary evidence, not a faithful record of the model's decision process.
- Sculley, D. et al. (2015). Hidden Technical Debt in Machine Learning Systems. NeurIPS 28.
  Monitoring, feedback loops, and configuration debt — written for ML pipelines, true for agents.
- Regulation (EU) 2016/679 (GDPR), esp. Art. 5 (minimisation, storage limitation) and Art. 17 (erasure).
  Why full-payload traces are a compliance object, not just an engineering convenience.